

Hacker Tools

June 20, 2026

Ramasamy Kandasamy

1. Command Line Tools

Files & Directories

<code>cd</code>	Change directory.
<code>pwd</code>	Print working directory
<code>ls</code>	Options: <code>-l</code> <code>-lh</code>
<code>tree</code>	List in tree form. eg: <code>tree dir</code>
<code>touch</code>	Creates text file.
<code>mkdir</code>	Make directory
<code>mkdir -p</code>	Make directory and necessary parent dir.
<code>cp</code>	Copy files.
<code>mv</code>	To move files and rename files.
<code>rm</code>	Remove files permanently.
<code>rm -i</code>	Remove files interactively.
<code>rm -r, rm -R</code>	Remove files recursively. Use to delete folders.
<code>rm -f</code>	Force delete.
<code>basename</code>	Removes folder name from path and optionally suffix.
<code>-s</code>	Remove suffix. eg: <code>basename -s .fastq <path></code>
<code>~</code>	Home directory, aka <code>\$HOME</code> .
<code>./, ../</code>	Relative paths to current and parent dir.
<code>/dev/null</code>	Fake file, black box.
<code>chmod 777</code>	r-4,w-2,x-1. User, group,all.
<code>chmod xyz</code>	Eg <code>chmod u+w</code> . x = u : user, g : group, a : all. y = + : add, - : remove. z = r : read, w : write, x : execute.
<code>du</code>	Disk usage, i.e., size of blocks allocated.
<code>-s</code>	Summarize, show total only.
<code>-h</code>	Human-readable sizes.
<code>-b</code>	Actual file size in bytes.
	Equivalent to <code>--apparent-size</code> , except in bytes.
<code>df -h</code>	Gives information about disk usage.

File compression

<code>tar</code>	Tape archive. Combine with a compression flag to handle <code>.tar.gz/.tgz</code> , <code>.tar.bz2</code> , <code>.tar.xz</code> .
<code>-c</code>	Create archive.
<code>-x</code>	Extract archive.
<code>-t</code>	List contents.
<code>-v</code>	Verbose.
<code>-f</code>	Use archive file (must be last flag, followed by filename).
<code>-z</code>	Filter through <code>gzip</code> , for <code>.tar.gz / .tgz</code> .
<code>-j</code>	Filter through <code>bzip2</code> , for <code>.tar.bz2</code> .
<code>-J</code>	Filter through <code>xz</code> , for <code>.tar.xz</code> .

<code>zip -r</code>	Compress. <code>zip -r file.zip dir</code>
<code>unzip -l</code>	View contents. <code>unzip -l file.zip</code>
<code>unzip</code>	Decompress. <code>unzip file.zip</code>
<code>gzip / gunzip</code>	Compress a single file to <code>.gz</code> , <code>.bz2</code> , <code>.xz</code> resp. (tar a directory first).
<code>bzip2 / bunzip2</code>	
<code>xz / unxz</code>	<code>gzip foo.tar → foo.tar.gz</code> (replaces file). <code>gzip -c foo.tar > foo.tar.gz</code> . <code>gunzip foo.tar.gz → foo.tar</code> .

TODO: `chown`, `chgrp`, `(un)compress`, `zgrep`, `zcat`, `zless`, `zdiff`.

Process Execution

<code>Cmd1 ; Cmd2</code>	Run <code>Cmd2</code> irrespective of exit status of <code>Cmd1</code> .
<code>Cmd1 Cmd2</code>	Execute <code>Prog2</code> only if <code>Prog1</code> has failed (non-zero exit status).
<code>Cmd1 && Cmd2</code>	Execute <code>Prog2</code> only if <code>Prog1</code> has succeeded (zero exit status).
<code>(...;...) ...</code>	Subshell: Both commands separated by a semi-colon are processed independently and piped in parallel to next step.
<code><(...)</code>	Process substitution, like anonymous named pipe. Eg: <code>program --in1 <(...) --in2 <(...)</code> .
<code>>(...)</code>	Write output to anonymous named pipe. Eg: <code>program --out1 >(...) --out2 >(...)</code> .
<code>xargs</code>	Execute command from stdin. Examples: - Apply <code>wc</code> on each file. <code>ls *.txt xargs wc</code> - Apply <code>wc</code> on each file, using placeholder. <code>ls *.txt xargs -I {} wc {}</code>. - List all files in each dir, with the dirname. [Two ways.] <code>ls xargs -I {} sh -c 'echo {}'; ls {}</code> <code>ls xargs -I {} sh -c 'echo \$1; ls \$1' - {}</code>
<code>source</code>	Execute a script in the current shell rather than in a new subshell.
<code>.</code>	Same as <code>source</code> .
Eg: <code>. foo.sh nohup</code>	Run a program without interruption.
<code>&</code>	Run in background. eg: <code>nohup prog1 &</code>
<code> </code>	Pipe
<code>tee</code>	Eg: <code>prog1 in.txt tee intermediate.txt prog > result.txt</code>
<code>mkfifo</code>	Create a named pipe. Eg: <code>mkfifo fqin</code> . Treat named pipe like any other file. But the input and output is piped. While using named pipe nothing is written on the disk.
<code>>, >></code>	Write and append, respectively, standard output to a file.
<code>2>, 2>></code>	Write and append standard error to a file.
<code>2>&1</code>	Redirects <code>std.err</code> to <code>std.out</code> .
<code><</code>	Take input.
<code>/dev/null</code>	Eg: <code>foo > /dev/null</code> , the output is not printed.

Process magement

<code>jobs</code>	List all jobs. Use <code>id</code> in <code>[]</code> to <code>bg,fg,kill</code> .
<code>fg</code>	Bring a job to foreground.
<code>bg</code>	Resume a suspended process in the background.
<code>Ctrl + Z</code>	Pause a running job.
<code>Ctrl + C</code>	Kill a running job.
<code>kill</code>	End a job.
<code>echo \$?</code>	Exit status,=0 when a program exits without an error.
<code>top</code>	Display tasks and system resource usage.
<code>htop</code>	User friendly tool to view running processes and resource utilization.

Terminal customization

Generally included in the `.bashrc` file.

- Alias**
Store new commands.
`alias foo="..."`
- DIRTRIM**
Set number of parent dirs displayed in the terminal.
`PROMPT_DIRTRIM=1`
This setting results in the display of only the immediate parent directory.
- Add to path.**
Eg: `export PATH=$PATH:~/local/opt`
- Run a script at the beginning**
Eg: `source /home/user/catkin.wc/devel/setup.bash`

Etc	
find	Usage: <code>find <folder> -name "<pattern>".</code> Eg: <code>find . -name foo.sh.</code>
-name <pattern>	Find <pattern> using same special characters as bash (*,?, [...])
-iname	Identical to -name but case-insensitive.
-empty	Matches empty files and folders.
-type <x>	Matches types x (f - file, d - directory, l - links).
-size <size>	Matches <size>. Eg: +50M ; Files larger than 50 MB Eg: -50M; Files smaller than 50 MB
-regex	Match regular expression. Use -E for extended POSIX.
-iregex	Case-insensitive.
rsync	Sync files from source to target. Usage: <code>rsync <options> <source-dir> <target-dir></code> Eg: <code>rsync -av sdir tdir.</code> Copies the directory sdir into tdir. tdir now contains sdir. Eg: <code>rsync -av sdir/ tdir.</code> tdir now contains the contents of sdir and not the directory sdir itself.
-av	Most common options. Sufficient for most use cases.
-avn	Dry run.
-delete	Delete files in the target dir that are not in the source dir.
?,*, [A-Z]	Wild cards.
{ }	Expands combinatorially. Eg: <code>\$ mkdir dir-{1, 2, 3}</code> Eg: <code>\$ mkdir dir-{1..100}</code> Eg: <code>touch foo-{A..C}-{1..10}</code> Eg: <code>echo "...\${(....)}..."</code>
\$()	Eg: <code>mkdir results-\$(date +%F)</code> Eg: <code>\$ today = "date +%F".</code>
export	Exports variable to child processes.

2. System Tools

df -h	View usage of all the mounted disk.
free -h	RAM usage.
uname -a	Info such as kernel name, architecture, version etc.
lspci	Lists all the PCI devices.
lsusb	Lists USB devices.
lscpu	List CPU info.
lsblk	List block devices like HDD, SSDs and Partitions.
lshw	List all the hardware info about the PC.

2.1. Important Directories.

bin	Essential binaries like, ls, cp, etc.
boot	Files related to boot process.
dev	Hardware and virtual devices. • /dev/sda : Storage devices. • /dev/tty : Terminal devices.
etc	Configuration files and directories. Eg: /etc/passwd
lib	Shared libraries.
media	Mount points for automatic mounting of removable media like pendrive, CDS etc.
mnt	Temporarily mounting filesystems such as external drives.
opt	Individual applications. Eg /opt/microsoft-edge.
usr	User specific files.
var	Stores data that changes frequently. /var/cache. /var/log. /var/lib.

Directories in **usr**. Shares some similarity with root: **bin**, **lib**.

usr/...	
include	Header files for development. Used during compilation.
local	Software installed manually, such as custom compiled application.
share	/usr/share/man. /usr/share/doc. /usr/share/icons. /usr/share/keyrings.

- **/etc/apt/preferences**
Manage package version and pinning, to ensure certain packages come from a specific repo and to prioritize certain versions.
- **/etc/apt/sources.list.d**
Manage third party repositories. New repos can be added using `add-apt-repositories` or manually.
- **/var/cache/apt/archive**
Holds .deb packages downloaded by apt.
- **/usr/share/keyrings**
Contains GPG keyrings used by apt to verify the authenticity of packages and repositories.

- **~/local/**
User specific software and files.
- **~/local/share/**
User specific data files.
- **~/local/share/applications**
Contains desktop entry files.

2.2. Package management.

which	Show where a executable is located. Eg: <code>which vim.</code>
ubuntu-drivers	• ubuntu-drivers autoinstall. Installs the best available drivers for the hardware. Especially useful after fresh install of Ubuntu. • ubuntu-drivers devices. Lists all the available drivers for your hardware.

Apt

apt	Command-line utility for managing package. Often requires <code>sudo</code> access.
install	• sudo apt install vim • To install a specific version: sudo apt install <pkg>=<ver>. Eg: <code>sudo apt install vim=1:8.2.3995-1ubuntu2.1</code> • --fix-broken. To fix broken dependencies.
update	Updates the local package index. Does not update or upgrade anything.
upgrade	Upgrades packages to the latest version based on the updated local package index.
remove	Uninstalls a package but leaves the configuration files intact.
purge	Uninstalls a package along with its configuration files.
autoremove	Uninstalls packages that were installed as dependencies but no longer needed.
autoclean	When a package is downloaded using apt the .deb file are stored in <code>/var/cache/apt/archives/</code> . autoclean remove the .deb files of packages that are obsolete.
clean	clean is like autoclean , but it removes all the .deb files irrespective of whether the package is obsolete or not. This clears out everything in <code>/var/cache/apt/archives/</code> .
list	List available packages. --upgradable , shows packages with available updates. --installed , shows installed packages.
search	Search for a package in the database. NOTE: unlike list, search looks for a keyword in the package name and description etc. ∴ this is not the same as say <code>apt list grep <keyword></code> .
show	Show detailed info of a package, including its dependencies.

Add a repository to apt

- Add repo using add-apt-repository from the software-properties-common package.
Eg: `sudo add-apt-repository "deb http://example.com/repo/ubuntu focal main"`
- **Add a repo manually** by editing the `/etc/apt/sources.list.d/example.list` file
Eg: `sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'`
- **Add gpg keys** using apt-key.
Eg: `curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -`
- Use `sudo apt update` to include the new repository's packages and then install the package.
Eg: `sudo apt update`
`sudo apt install ros-noetic-desktop-full.`

DPKG

Install .deb files. dpkg is a low level package manager and unlike apt it does not handle dependencies.

When there is some issue with dependencies after installing using dpkg, `sudo apt install --fix-broken` might help.

- `-i, --install:`
Install a .deb package.
- `-r, --remove:`
Uninstall but retain the config files.
- `-P, --purge:`
Uninstall package and remove the related config files.
- `-l, --list:`
List all the packages.
Output: First column indicates the status (intended actions, current status).

3. Networking

3.1. Basic Tools

- **ifconfig.** Lists network properties like IP addresses.
Lists info regarding loop-back, ethernet and WiFi (etc.)
 - `lo` is loop back.
 - `inet:` IPv4.
 - `inet6:` IPv6.
 - `ether:` MAC address.
 - `RX:` Stats on received data.
 - `TX:` Stats on transmitted (sent) data.
- **ip addr show.** Similar to ifconfig. Show only the IP addresses and not the stats.
- **ping.** Eg: `ping google.com`
Give TTL, RTT and packet loss when connecting to the host.
- **hostname**
`hostname -I` Gives all the IP addresses.

3.2. Cryptography

SSH

- **Generating the keys.** Examples:
`ssh-keygen -t ed25519 -C "your_email@example.com"`
`ssh-keygen -t rsa -b 4096 -C "your_email@example.com"`
 - t: Algorithm to use.
 - b: Key length.
 - C: Label for the key, generally email ID.
- Public keys: `~/.ssh/id_rsa`
- Private keys: `~/.ssh/id_rsa.pub`
- Test authentication: `ssh -T gitgithub.com.`

GPG Keys

- **Generate key:** `gpg --full-generate-key`
Follow the instructions to generate the key.
- `gpg --list-keys.`
- **Export public key:**
`gpg --armor --export your_email@example.com > public_key.asc`
`--armor` Output key in ASCII format.
`--export` Specify the key.
- **Import key:**
`gpg --import public_key.asc`
- **Sign a file:**
`gpg -sign file.txt`
- **Encryption / Decryption:**
Encrypt:
`gpg --encrypt --recipient recipient_email@example.com file.txt`
Decrypt:
`gpg --decrypt file.txt`
- Keys are located in `~/.gnupg` directory.

3.3. File Transfer

<code>wget url</code>	Download file from http or ftp.
<code>-- accept, -A "..."</code>	Only download files matching this criteria. Eg <code>"*.fastq"</code>
<code>-- reject, -R</code>	Similar to above
<code>--no-directory, -nd</code>	Don't download directory structure.Only files.
<code>--recursive, -r</code>	
<code>--no-parent, -np</code>	Don't move above parent directory. This is important to avoid downloading unnecessary data.
<code>-O</code>	Output filename.
<code>-e robots=off</code>	To not want wget to follow 'robot.txt'. See: This answer

Other options: `--limit-rate, --user=user, --ask-password`

<code>curl url > file</code>	Redirect output to file.
<code>curl -O <file></code>	download to file.
<code>-L,--location</code>	Download ultimate page and not the redirect page.

Curl can also download form SFTP and SCP. Also checkout RCurl and pycurl.

3.4. Remote machines

Login with SSH

- **Usage**
\$ ssh host
\$ password:
- **Examples of host**
192.162.82.120
hpc.myuniversity.edu
turing@192.162.82.120
turing@cse.univ.edu
- **Options**
-v verbose. Verbosity can be increased by: -vv or -vvv.
-p port. Eg: ssh -p 5043 aturing@cse.univ.edu
Default port is 22
- **Using alias:** To use alias create the file ~/.ssh/config and store server as info as below. Host hpc_serv
HostName 190.257.170.129
User aturing
Port 50434
Also applies for Rsync and scp

TODO: scp.

3.5. Setting up a server

Use "Open SSH":
<https://help.ubuntu.com/lts/serverguide/openssh-server.html>

4. Text processing

echo	Process and print whatever follows.
echo -e	enable backslash escapes like \\, \t, \n
cat	Takes standard input or input from file and gives standard output.
cat -n	Output with line numbers.
head -n x	Print first x lines.Default: 10 lines.
tail -n y	Print last y lines.
wc	Word count. Outputs number of words, lines and characters.
wc -l	Outputs only number of lines.
tr	Translate. Eg: tr ':' '\t'.
less	Pager. Commonly used commands:
Space	Next page.
b	Previous page.
g	First line.
G	Last line.
j	Down (One line at a time).
k	Up (One line at a time).
/<pattern>	Search down for a pattern.
?<pattern>	Search up for pattern.
n	Repeat last search downward.
N	Repeat last search upward.
cut	To extract specific columns.
-f x	Extract columns x.
-f x-z	Extract range of columns.
-f w,x-z	Extract w and x-z. Cut cannot reorder column.
-d	Specify delimiter eg: -d", ". Default delimiter is tab.
column -t	To visualize columns of data. Usually data is piped to column -t.
-s	Specify delimiter using -s", ". Default: tab.
grep	Use as grep "<pattern>" file . Quotation around the pattern is not necessary but it is safe. If the pattern contains quote then use single quotes eg: grep '..."..."' .
-i	Case insensitive.
-E	To use regular expressions in grep.
^	Look for pattern in the beginning of line. Eg: "^#"
-w	Matches the entire word surrounded by space.
-v	Returns only lines that do not match the pattern.
-o	Return the exact matching pattern.
-c	Count how many lines match a pattern.
-B1	Print one line of context before the matching line.
-A2	Print two lines of context after the matching line.
-C	Context before and after the matching line.(Doesn't work?)

sort	Sorts alphanumerically by line.
-ka,b	Sorts w.r.t to columns a to b.
-k2,2n	Treats columns 2 as numeric and sorts w.r.t to columns 2.
-t	Specify delimiter eg: -t", ". Default = tab.
-s	Stable sort. Do not reorder lines in file if the sort rank is equal.
-c	Check if the file is already sorted.
-r	Reverse sort.
-V	Understands numbers inside string. Eg chr22.
-S	Specify memory to be used. Eg: -S 2G # Use 2 GB, -S 50% # Use 50% of memory.
--parallel	to use parallel processing.
uniq	Usually used along with sort as : sort ... uniq.
-i	Case insensitive.
-c	Count occurrences next to the unique lines.
-d	Return line with duplicates.
join	Combine data based on a common column. Eg: join -1 a -2 b file1 file2. a and b represent two columns common to file1 and file2.
-a	If some elements of common column are missing from one file. Use this flag to show all elements of common column from superset file.
diff	Compare two text files. diff file1.txt file2.txt . First line in output indicates the kind of change. a added newline, d deleted a line, c change in the line. After diff echo \$? is 0 if there is no difference.
-b	Ignore changes in white spaces.
-w	Ignore all blank spaces. Does not ignore blank lines.
-B	Ignore blank lines. The above two do not ignore blank lines.
-Z	Ignore trailing white spaces.
cmp	Compares two files byte-by-byte and outputs the first byte that differs.
md5sum	md5sum <input-file>
shasum	Calculate checksum using SHA-1. Can be used to find checksum of many files and store the result in a text file. Eg: shasum *.fa > chksum.sha
-c	Validate the files. Eg: shasum -c chksum.sha .
sum	Checksum program used by Ensemble.
diff -u	Outputs a diff file that shows difference between two files. Eg: diff -u file1 file2

4.1. Awk

Format: <code>awk pattern {action} input1.txt input2.txt</code>	
<code>awk -f file.awk input.txt.</code>	
Record = row. Column = fields.	
<code>-F</code>	Input field separator. Eg: <code>awk -F",", "input.txt.</code> Defaule field separator = tab.
<code>-f</code>	Take input from file. Eg: <code>awk -f file.awk input.txt.</code>
<code>(...) && (...)</code>	Use logical operators. See below.
<code>\$n /.../</code>	Use regular expression between slashes.
<code>/.../,.../</code>	Specify range. Works only with regex (with double slash) .
<code>BEGIN{...}</code>	Eg: <code>awk 'BEGIN{...} ... {...} END{...}'</code>
<code>END{...}</code>	

Awk operations: +, -, *, /, %, ^.

a ... b. Replace "...": ==, !=, <, >, <=, >=, ~, !~, &&, ||, !a

Field separators: FS, RS, OFS, ORS.

Awk variables: NF, NR (Record number accumulates between files.), FNR (Resets record number after every files.).

```
Example awk script file
awk -f script.awk plasmids.tsv
BEGIN{FS="\t";OFS="\t";x=0}
/[Cc]re/{
x+=1;
print x,$1,$2}
END{print "There are " x "plasmids with Cre"}
hline
```

[Checkout BioAwk.](#)
[Checkout control flow.](#)

4.2. Sed

<code>sed 's/target/replacement/flag'</code>	
<code>-e</code>	to Chain commands.Eg: <code>sed -e 's/:/~/ -e 's/-/~/</code> .
<code>-E</code>	Use extended POSIX.
<code>g</code>	Global flag. Usually sed replaces only the first occurrence in a sentence. Use global flag to replace all occurrences.
<code>i</code>	To make the search case insensitive.

4.3. Regular Expression

Single character meta characters	
<code>.</code>	Match any single character.
<code>[]</code>	Match any single character between []. Eg: <code>[at]</code> match "a" or "t".
<code>[^]</code>	Match any single charcter except on between [].
<code>[0-9]</code>	Any number between 0 and 9. Eg: <code>0-3a-cz]</code> equals <code>[123abcz]</code> .
<code>(...)</code>	Grouping. eg: <code>(AT)+</code> or <code>(GLY) {2,}</code> .

Quantifiers	
<code>?</code>	Match preceding character zero or one time.
<code>*</code>	Match zero or more time.
<code>+</code>	Match one or more time.
<code>{n}</code>	Match n times.
<code>{n,}</code>	Match atleast n times.
<code>{a,b}</code>	Match atleast a times, atmost b times.

 Anchors	
<code>^</code>	Match the start of a line.
<code>\$</code>	Match end of a line.
<code>\<</code>	Match beginning of word.
<code>\></code>	Match at the end of word.
<code>\b</code>	Match either beginning or end of word.
<code>\B</code>	Match any character not at the beginning or end.
Character class	
<code>[alnum:]</code> , <code>[digit:]</code> , <code>[alpha:]</code> , <code>[upper:]</code> , <code>[lower:]</code> , <code>[blank:]</code> , <code>[space:]</code> , <code>[punct:]</code> and <code>[print:]</code> .	
Use backslash as escape character.	
<code>\s</code>	white space character. What it includes depends on the flavour of regex.
<code>\d</code>	Add digits.
<code>\w</code>	Word character, matches <code>[A-Za-z0-9_]</code>
as OR logical operator: (GLY GLN). "one and or two" is equal to "(one and) (or two)".	
"one (and or) two" is "one and two" or "one or two".	
Back references: () : Memorizes the match for regular expression within parenthesis. Use <code>\n</code> to recall nth match.	

5. Shell scripting

Modifying PATH

Add a directory to path: Append one of the following files.
~/.profile or ~/.bash_profile
with the following line:
`PATH=$PATH:<directory>`
Eg: `PATH=$PATH:$HOME/scripts`

Header

<code>#!/bin/bash</code>	Shebang
<code>set -e</code>	Terminates script if there is non-zero exit status.
<code>set -o pipefail</code>	If a program in the pipe fails the entire pipe returns non-zero exit status.
<code>set -u</code>	Terminates for undefined variables.

Variables

<code>sample="CNTRL"</code>	Assignment, no space around "="
<code>echo \$sample</code>	
<code>echo \${sample}_aln</code>	Use curly braces while concatenating a variable with additional text.
<code>mkdir "\${sample}_aln"</code>	Quoting variables prevents commands from interpreting spaces and special variables.
<code>echo \${#sample}</code>	Length of the variable sample

Command-line arguments

<code>\$0</code>	Script name
<code>\$1</code>	First argument
<code>\$n</code>	n^{th} argument.
<code>##</code>	Number of arguments not including \$0.

Example:

```
#!/bin/bash
echo "script name: $0"
echo "first arg: $1"
echo "second arg: $2"
echo "There are $# input arguments"
```

5.1. Conditionals

Format

```
if [ <conditon-statement> ]
then
if-statements
elif
then
elif-statements
else
else-statements
fi
```

Example:

```
if [ $# -lt 3 ]
then
echo "There are less than 3 arguments"
fi
```

In bash 0 is true/success, anything else is false/failure

String and integer comparison

```
-z str      str is null string.
str1 == str2 str1 and str2 are identical.
str1 != str2
int1 -eq int2 int1 and int2 are equal.
int1 -ne int2
int1 -lt int2
int1 -gt int2
int1 -le int2
int1 -ge int2
-o          Logical OR.
-a          Logical AND.
```

if conditional can also be used to depend on exit status. Eg:

```
if grep "pattern" file1.txt > /dev/null && grep
"pattern" file2.txt > /dev/null/
then
echo "found pattern in file1.txt and file2.txt"
fi
```

```
if ! grep "pattern" file1.txt > /dev/null
then
echo "pattern not found in file1.txt"
fi
```

Testing files and dirs

List of test expressions.

```
-d dir    dir is a directory
-f file   file is a file.
-e file   file exists.
-h lind   link is a link.
-r file   file is readable.
-w file   file is writable.
-x file   file is executable.
```

Example

```
test -d dir ; echo $?

test -d dir1 -o -d dir2; echo $?
```

Exit status would be 0 if the directory dir exists.

Example:

```
if ! test -d $1
then
mkdir $1
fi
```

Above script is equivalent to the following.

```
if [ ! -d $1 ]
then
mkdir $1
fi
```

5.2. Arrays and For loop

Manual creation

```
$ sample_names=(zmayA zmayB zmayC)
$ echo ${sample_names[0]}
zmayA
$ echo ${sample_names[@]}
zmayA zmayB zmayC
$ echo ${#sample_names[@]}
3
$ echo ${!sample_names[@]}
0 1 2
```

Array creation using command substitution

```
samples=$(cut -f3 samples.tsv)
file_names=$(ls)
```

Array of number sequence

```
seq 0 0.1 1 # seq start step end
s=$(seq 0 0.1 1)
```

<code>\${arr[i]}</code>	(i-1) th element of <code>arr</code> .
<code>\${arr[@]}</code>	All the elements of <code>arr</code> .
<code>\${#sample_names[@]}</code>	Length of <code>arr</code> .
<code>\${!sample_names[@]}</code>	Returns an array containing the index of elements in <code>arr</code> .

5.3. For loop

```
for name in ${file_names[@]}
do
process.sh $name
done
```

```
for name in ${file_names[@]}; do
process.sh $name
done
```

```
for name in ${file_names[@]}; do; process.sh $name; done
for i in $(seq start step_size end);
do
process.sh $i
done
```

5.4. Find, exec and xargs

<code>expr -and expr</code>	Logical AND.
<code>expr -or expr</code>	Logical OR.
<code>-not expr</code>	Logical NOT. Alternate: "!" <code>expr</code>
<code>(expr)</code>	Group a set of expressions.
<code>-exec</code>	Example: <code>find . -name *.c -exec <prog1> {} \;</code> Execute <code><prog1></code> on all the found files. <code>{}</code> represents the found files. Mind the space between <code>{}</code> and <code>\;</code>

5.5. Arithmetics

let

Examples using `let`:

```
let x=1 #No space within expression
let x=x*2
let x++
let "x = x + 1" # Space OK within quotation.
```

Examples using `expr`:

```
expr 2 + 3 # Space is required for expr
a=$(expr 2 + 3)
expr $x + 1
```

`expr` is similar to `let`, but only evaluate and not assign value to a variable.

Arithmetic operations:

<code>+</code> , <code>-</code> , <code>/</code> , <code>%</code>	
<code>*</code>	Multiplication operator for <code>let</code>
<code>/*</code>	Multiplication operator for <code>expr</code>
<code>var++</code>	increment var by 1 used only in <code>let</code>
<code>var--</code>	increment var by 1 used only in <code>let</code>

6. Git

Configuration and Initialization

<code>git init</code>	Intialize empty repo. Creates <code>.git</code> dir.
<code>git config</code>	
<code>--global</code>	Global scope. <code>/.gitconfig</code> .
<code>--system</code>	All users. <code>/etc/gitconfig</code> .
Default	Repo. <code>.git/config</code> .
<code>--list</code>	List all the config.
<code>--edit</code>	Manually edit the config.
Common configs.	
<code>user.name</code>	String.
<code>user.email</code>	String.
<code>color.ui</code>	Boolean: true / false.
<code>core.editor</code>	String. Eg: vim, gedit.

Working with Remotes

<code>git clone</code>	Clone a repo.
<code>git remote</code>	
<code>-v</code>	Show remote repo linked to local repo. Also see: <code>git branch -vv</code> .
<code>add</code>	Add a remote repo. Eg: <code>git remote add <path/to/remote/repo></code> .
<code>rm</code>	Remove remote repo. Eg: <code>git remote rm origin</code> .
<code>git fetch</code>	Get info regarding changes in the remote. <code>git fetch origin</code> .
<code>git pull</code>	Pull changes from remote branch. Eg: <code>git pull origin feature</code> .
<code>git push</code>	Push changes from remote branch. Eg: <code>git push origin feature</code> .
<code>-u</code>	Push a new branch to remote <i>and set upstream tracking</i> .
<code>--force</code>	Necessary when the history differs between origin and local, eg. after <code>--amend</code> or <code>git reset</code> . Safer: <code>-force-with-lease</code> : Pushes only if no one else has updated the branch.

`git submodule`

Basic Workflow.

Common themes:

- `.` (dot) represents all files.
Eg: `git add .` stages all the files.
 - `--` Names following this list the target file(s).
Eg: `git log -- foo1.txt`
 - `-p` Patchwise option for making changes.
 - `y / n` Stage or do not stage the hunk.
 - `j / J` Go to next hunk (staged j).
 - `k / K` Go to previous hunk (staged k).
 - `s` Split the hunk.
 - `/` Search for a hunk matching a regex.
 - `q` Quit without staging remaining hunks.Applies with `git add`, `git stash`, `git restore`.
- TODO:** What are other command with '-p' option?

<code>git add</code>	Start tracking a file or stage a file for commit.
<code>-f</code>	To stage a file not tracked, i.e. a file in <code>.gitignore</code> .
<code>git commit</code>	Commits all staged files to repository.
<code>--amend</code>	Amend to last commit. Eg: <i>Deleting a file from previous commit.</i> <code>git reset --mixed</code> <code>git add <selected-files></code> <code>git commit --amend</code> . NOTE: Also useful to add files to last commit and edit the latest commit message.
<code>-a</code>	Commit all tracked files.
<code>-m "..."</code>	Message is mandatory. If there is no message, git opens text editor to input message. Default text editor can be specified in <code>git-config</code> .
<code>git reset</code>	Undo commit. Eg: <code>git reset --soft HEAD~</code> <code>--soft</code> Keep changes on stage. <code>--mixed</code> Keep changes unstaged. <code>--hard</code> Delete changes.
<code>git revert</code>	Eg: <code>git revert <c1></code> . Undo changes introduced after <code><c1></code> , but keep the commits and create a new commit.
<code>git restore</code>	Undo changes in the tracked but not staged files.
<code>--staged</code>	Unstage staged changes.
<code>git clean</code>	To remove untracked files / directories.
<code>-n</code>	Dry run.
<code>-f</code>	<i>force</i> Remove only untracked files.
<code>-fd</code>	Remove untracked files and directories. NOTE: <code>git clean</code> requires <code>-f</code> option. Without <code>-f</code> no files are deleted.
<code>git stash</code>	Stash changes with default message.
<code>-u</code>	Stash untracked changes.
<code>push</code>	<code>-m</code> : Stash message. <code>--keep-index</code> : Stash only unstaged changes.
<code>apply</code>	Apply a stash but do not remove it from the stash.
<code>drop</code>	Drop a stash from the stack.
<code>pop</code>	Apply a stash and remove it from the stack.
<code>show</code>	List the files in a stash.
<code>show -p</code>	List the changes in the files in the stash.
<code>list</code>	List all the stashes.
<code>clear</code>	Drop all the stash.
<code>branch</code>	Create a branch from a stash. Eg: <code>git stash branch b1 stash3</code> .

Branching and Navigation

<code>git branch</code>	Create a branch. <code>git branch <branch-name></code> <code>git branch <branch-name> <commit></code> <code><commit></code> is an existing commit. It could be and unreachable commit.
<code>-f</code>	<code>git branch -f b1 <commit></code> . Force a branch to point to a target commit. <code><commit></code> could be another branch.
<code>-a</code>	List all branches. A.k.a - <code>--all</code>
<code>-m</code>	Rename a branch. Eg: <code>git branch -m <new-name></code> Rename current branch to <code>new-name</code> . Eg: <code>git branch -m old new</code> Aka: <code>--delete</code> . Delete merged branches.
<code>-d</code>	Force delete a branch.
<code>-u</code>	Set upstream tracking. Eg: <code>git branch -u origin/b1 b1</code> .
<code>-vv</code>	Show local branches and corresponding remotes.
<code>git tag</code>	<code>git tag <tag> <commit></code> . <code>git tag <tag></code> . Annotated tag. <code>git tag -a <tag> <commit> -m "..."</code> . Delete a tag.
<code>-d</code>	Delete a tag.
<code>-l</code>	List all tags.
<code>--edit / -e</code>	Edit tag annotation.
<code>git merge</code>	<code>git merge <other-branch></code> Merge other to current. <code>git merge b1 b2</code> : Merge b1 to b2.
<code>--abort</code>	Abort a merge in progress.
<code>git rebase</code>	Apply changes from a branch to the target. Eg: <code>git rebase master</code> : Apply changes in current branch on top of master. Eg: <code>git rebase master dev</code> : Apply changes in dev on top of master. Eg: <code>git rebase master <commit> dev</code> . Eg: <code>git rebase master <from> <to></code> .
<code>--abort</code>	
<code>git checkout</code>	Navigate to a commit.
<code>-</code>	Checkout to previous commit. Eg: <code>git checkout -</code>
<code>-b</code>	Create a new branch checkout to it. Equivalent to <code>git branch</code> and <code>git checkout</code> .
<code>-- <file></code>	Restore a file from a commit. Eg: <code>git checkout branch1 -- foo.txt</code> . Eg: <code>git checkout commit1 -- foo.txt</code> .

Inspection tools

<code>git status</code>	
<code>-s</code>	Short format.
<code>git diff</code>	Diff between working state and [default]. Eg: <code>git diff <old> <new></code> . Eg: <code>git diff HEAD : HEAD</code> vs working. Eg: <code>git diff <commit> : <commit></code> vs working.
<code>--staged</code>	Diff between staged and HEAD.
<code>--cached</code>	Same as <code>--staged</code> .
<code>--name-only</code>	List only the diff file names.
<code>--</code>	Diff for specific files. Eg: <code>git diff -- foo.txt</code> .
<code>git log</code>	Show commit history.
<code>-nx</code>	Show last <code>x</code> commits. <code>n</code> can be ignored. Eg: <code>-10</code> .
<code>--oneline</code>	Condense message to oneline, shorten hash.
<code>--graph</code>	Show as graph.
<code>--decorate</code>	Display tags and branch names.
<code>--all</code>	Show all branches, and stash? .
<code>--branch</code>	Show all local branches. No remote and stashes.
<code>--remotes</code>	Show all remotes branches, and no local?
<code>git fsck</code>	File system check.
<code>--unreachable</code>	List all unreachable commits.

Selected Commands

- `git rev-parse`: Show commit hash.
Eg: `git rev-parse HEAD`.
Eg: `git rev-parse <commit>`.
`--short` : Display shorter hash.
`--abbrev-ref` : Display branch name.
- `git check-ignored <file>`: Check whether the file is excluded.
`git check-ignore -v <file>`: *Verbose* - also tells which pattern in `.gitignore` causes the file to be excluded.
- **Files**: Use `git rm`, `git mv`; instead of `rm` and `mv`.
- `git ls-tree`: List cont

TODO: `git ls-tree`.

`cherry`.

`git remote prune origin`.

`git rm --cached`.

`git check-ignore -v !file`.

`git reflog`.

`git config` with an example config with alias.

Aliases like `HEAD`, `HEAD`; `HEAD@{1}`, `stash@{1}`.

`squash??`.

Patch file.

`git switch`.

`git show`.

Resolving merge conflict in terminal.

`git blame`.

`git stash -p` and `-a`.

GitHub

Authentication using SSH and GPG keys.

1. Add the public keys to your GitHub account.
2. Change remote URL to use SSH instead of HTTPS.
3. Change `.gitconfig` to use GPG scheme for commits:
`git config --global user.signingKey <key_id/email>`
`git config --global user.commit.gpgSign true`
4. Change the remote repo link to use SSH instead of HTTPS:
`git remote set-url origin`
`git@github.com:username/repository.git`

Gitignore

Store patterns in `.gitignore` file.

Patterns file/directory paths with matching pattern will be excluded from tracking.

Patterns

- ! Reinclude an excluded file. **See Note below**.
- / Directory separator.
- * Match anything except `/`.
- ? Match any single character except `/`.
- ** Match anything including `/`.
- \ Escape character.

NOTE: Files whose parent directory has been excluded will not be reincluded.

Eg:

- Pattern without a slash:
Match any file / directory with that name.
Eg: `foo` matches with the file/directory: `/bar/foo`, `foo`
- Pattern ending with a slash:
Matches only with directories.
Eg: `foo/` Does match a file.
- Pattern with slash in the beginning or middle:
Matches with path relative to `.gitignore`
Eg: `/bar/foo` does not match `/foo` or `/tmp/bar/foo`

```
$ mkdir foo && touch foo/tmp{1..5}.txt
----
/.gitignore
foo/*
!tmp1.txt # tmp1.txt will be tracked.
----
/.gitignore
foo
!tmp1.txt # tmp1.txt will not be tracked.
```

NOTE: There is no gap after `!`.

7. Vim

Motion	Usage: <num> <motion>
h l	One character left or right.
j k	One line up or down.
w b	One word forward or backwards.
e	Similar to w but keeps the cursor at the end of the word.
O	Cursor to the begining of the sentence.
\$	Moves cursor to the end of the sentences.
G	End of the file.
gg	First line.
H	Top of screen.
M	Middle of screen.
L	Botom of screen.
<num>G	Go to line <num>.
[Ctrl] + f	One screen forward.
[Ctrl] + b	One screen backward.
[Ctrl] + G	View position in the file.
[Ctrl] + O	Go to where you came from .
[Ctrl] + I	Opposite of [Ctrl] + [O]
%	Go to the corresponding opening or closing parenthesis.

Operators

i	INSERT mode
a	append, goes to insert mode
A	append from the end of the line.
v	visual selection, selection is stored in clipboard
o	open a line below
O	open a line above
[Esc]	Go to command mode
d	delete and also cut, $\equiv$ [Ctrl] + [X]
dd	delete whole sentence
x	delete character under the cursor
r	replace the character under the cursor
R	replace until [Esc]
c	change: works equivalent to d followed by i
y	yank, copy
p	paste
u	undo most recent edit
U	undo all the changes in the line
[Ctrl] + [R]	Redo

Copy, paste, bookmark

:xmy	Move line x below line y.
:x,ymz	Moves lines between and including x and y below line z.
:xty	Copy line x below line y.
:x,ytz	Copy lines between and including x and y below line z.
ma	Set bookmark at current line. a $\in$ [a-z].
'a	Jump to bookmark a.
: 'a, 'bco'c	Copy lines between and including bookmarks a and b below bookmark c.
: 'a, 'bco'z	Copy lines between and including bookmarks a and b below line z.

Search and replace

:/REGEX	Find regular expression.
n	next search target
N	Previous search target
:s/target/replace	Similar to sed. Replaces target only in the current sentence and only once.
:s/target/replace/g	Replaces at all instance in the current sentence.
:%s/target/replace/g	Replaces through the entire file.
:%s/target/replace/gc	Ask for confirmation at each instance.

Save, write and Exit

:q	quit
:q!	quit without saving
:w	save the current file
:wq or :x	save and quite
:w file	write to file.
:xyw file	write lines between and including lines x and y to file.
:!	Execute shell command. Eg: :!pwd

:set

Usage: :set option. Eg: :set ic
ic Case-insensititve search
hls Highlight search
number Show line number

To turnoff the option use no. Eg :set noic to turnoff ic

Etc

[Ctrl] + **[D]** for command completion.

[Tab] for filename completion.

For further setting: ~/.vimrc

Help:

[F1]

:help

:help w

:help user-manual

Default settings: Set default settings in ~/.vimrc

Create this file if it does not exist.

Example .vimrc file:

```
syntax on
colorscheme desert
set number
set hls
```

8. Markdown

Text formatting:

- **italics**
- ****bold****
- ***** bold italics *****
- __underline__
- __*underline italics*__
- __**underline bold**__
- __***underline bold italics***__
- ~~~~strikethrough~~~~
- Text coloring:
 blue text

Heading, lists and links

- Itemized list: * item 1 or + item 1 or - item 1
- Ordered list: Eg:
 1. red
 2. blue
 4. green # Here output automatically numbers it to 3
- Use # for Headers.
Header level 1
Header level 2
Markdown supports upto 6 levels.
- <http://website.com/link>
- [link text](http://website.com/link)
- Insert figure
![alt text](path/to/figure.png/)

Inserting code

- ‘inline code’, Use backticks.
- Code block with tilde:

```
~~~ Language (Optional used by pandoc to )
code block
code block
~~~~~
```
- Codeblock with three backticks:

```
““Language (Optional used by pandoc to )
code block
code block
code block
““
```

9. Pandoc

- **Markdown to HTML (simple version)**
\$ pandoc -f markdown -t html README.md -o README.html
- **md to word**
\$ pandoc -s README.md -o README.docx
- **Standalone:** -s. Necessary for syntax highlighting.
To get list of languages: --list-highlight-languages

- **Box/shading for code:** Use `--highlight-style`. Eg:
`--highlight-style tango` # Good for light shade.
`--highlight-style breezedark` # Good for dark shade.
`--list-highlight-style` # List of highlight themes.

10. Uncategorized

Terminal shortcuts

<code>ctrl</code>	+	<code>W</code>	Delete from cursor to beginning of word.
<code>ctrl</code>	+	<code>U</code>	Delete from current cursor to start of line.
<code>ctrl</code>	+	<code>A</code>	Move cursor to beginning of line.
<code>ctrl</code>	+	<code>E</code>	Move cursor to end of line.
<code>ctrl</code>	+	<code>L</code>	Clear the screen.
<code>alt</code>	+	<code>F</code>	Move forward by word.
<code>alt</code>	+	<code>B</code>	Move backward by word.

11. WSL and windows CMD

11.1. Execute command prompt commands from WSL.

- Notepad:
`notepad.exe`
`notepad.exe temp.txt`
- File explorer:
`explorer.exe`
`explorer.exe .`
- Execute command prompt commands in WSL.
`cmd.exe` *command-line-commands* Eg: Opening a windows program
`cmd.exe /C start program_name file_name`
Eg:
`cmd.exe /C start SumatraPDF.exe`
`mementopython3-english.pdf`

11.2. Open from command prompt

- Websites using edge or chrome.
Edge: `start microsoft-edge`
Edge: `start microsoft-edge:http://www.google.co.in/`
- MS-office apps.
- Other applications.

12. Using GUI in WSL

12.1. Installing XFCE

Under construction

Ref:

<https://www.youtube.com/watch?v=nKCe9UE-quA>

<https://www.shogan.co.uk/how-tos/wsl2-gui-x-server-using-vcxsrv/>

12.2. Running XFCE

Open XLaunch app

The following is just to open a windows with simple settings.

1. Double-click and open XLaunch app. You will see a dialog box for display settings.
2. Choose "One large window" and choose "-1" for Display Number. Click "Next".
3. Choose "Start no client". Click "Next".
4. Check "Clipboard", "Primary Selection", and "Native opengl". Click "Next".
5. Save the configuration if you want, or just click "Finish" to start the window.

Launch xfce in WSL

Execute the command `xfce4-session`. Ignore the warnings.

13. Incomplete:

NOTE: This cheatsheet does not include Bioconductor and GRanges. Ver2 has them. But I will split it to a different cheat-sheet, "Bioconductor and R"

- arithmetics in bash
- pandoc
- markdown syntax
- install packages
- make
- tabix
- SQL